

# ProofPrep

**AI placement prep built on one loop: diagnose → target → practise → re-measure.** Built for PromptWars × GenAI Innovation Challenge ("Reimagine Placement Preparation"). The full product plan is in [PLAN.md](#).

## The problem

Students at Indian engineering colleges prepare for campus placements (aptitude, DSA, CS fundamentals, HR and technical interviews) with generic question banks. They don't know **where they stand for a specific company, why** they get questions wrong, or whether their resume survives a technical interview. They practise more of the same and hope.

## What ProofPrep does

| Feature                           | What it does                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Quick Assessment</b>           | 10 aptitude, 10 coding and 10 communication questions. Every MCQ asks "how sure are you?" and records time. Communication answers can be typed or spoken (Web Speech API).                                                                                                                                                                                                                                                                                                  |
| <b>Skill Gap</b>                  | Overall, Coding, Aptitude and Communication rings. Your scores vs each target company's bar, weakest skills, a mistake-type breakdown, and AI suggestions for your role and companies.                                                                                                                                                                                                                                                                                      |
| <b>Roadmap</b>                    | Code computes a daily time budget from days left, hours per day, company weights and your gaps. The AI (P5) sequences it; task type follows your mistake type. Tick tasks off, regenerate any time.                                                                                                                                                                                                                                                                         |
| <b>Daily Practice</b>             | <b>Daily coding:</b> 3 problems a day for your domain and weakest skills, from 35 classics (LeetCode, Codeforces, HackerRank) restated in our own words with a link to the original. Write Python or JavaScript, <b>run the tests in your browser</b> (Pyodide/Web Worker, hidden tests on Submit), then get an AI review of complexity and edge cases. C++/Java get the AI review only. Plus a <b>concept drill:</b> 4 domain coding-pattern MCQs + 6 weak-spot questions. |
| <b>Mock Interview</b>             | Round 1: 10 timed MCQs (auto-submit at 10:00). Round 2: a coding problem with a 20-minute editor and an <b>AI code review</b> (not execution). Round 3: Project Defense. Final report.                                                                                                                                                                                                                                                                                      |
| <b>Projects</b>                   | Add a public GitHub repo and/or your resume. The AI checks every resume claim against the code: supported / partial / unsupported / unverified, with file links.                                                                                                                                                                                                                                                                                                            |
| <b>Placement Readiness</b>        | Per company, with the formula and arithmetic on screen.                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Progress</b>                   | Readiness per company and category scores over time (Recharts).                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Personal Details / Connect</b> | Edit your target; save GitHub, LinkedIn and LeetCode links and upload your resume.                                                                                                                                                                                                                                                                                                                                                                                          |

## Differentiators

1. **Mistake Diagnosis.** Every wrong answer is classified as a *concept gap*, *application gap* or *careless slip*, plus an *overconfident* flag when you marked it "Sure". Deterministic rules decide first (answer time, the misconception tagged on the option you chose, your mastery and concept-level accuracy). The AI (P4) only handles cases no rule covers, so every classification can be traced. The roadmap then prescribes by type: concept → learn, application → mixed drill, careless → timed drill, overconfident → calibration drill.
2. **Project Defense.** Your resume says "real-time with WebSockets"; your code polls every 10 seconds. ProofPrep reads the repo (README, tree, up to 8 key files), marks that claim **unsupported** with the file that shows it, and the interviewer asks about it first.
3. **Communication Coach.** Answers are scored on structure (STAR for behavioural, claim-reason-example for technical and opinion), relevance, clarity and length. Pace and filler words are computed in code. The AI rewrites *your own* answer in the right structure, with [placeholders] where facts are missing. It never judges accent.

## Readiness formula (transparent, verifiable)

For company  $c$  and components  $k \in \{\text{aptitude, coding/DSA, CS, projects, communication}\}$ :

$$\text{Readiness}_c = \sum_k w_{ck} \cdot \min(1, S_k / B_{ck}) \cdot 100$$

- $w_{ck}$ : company weights (sum to 1).  $B_{ck}$ : company bar. Reaching the bar earns full credit; there's no extra credit above it.
- Aptitude, DSA and CS: mean mastery  $\times 100$  of the skills you've attempted, weighted by the company's skill importance.
- Projects: per project  $50 \cdot (\text{supported} + 0.5 \cdot \text{partial}) / \text{claims} + 10 \cdot \text{defenseScore}$ , then the mean of your top 2 projects.
- Communication: mean `overall` of your last 3 answers, where `overall = mean(structure, relevance, clarity, length)  $\times 20$` .
- Confidence (low / medium / high) per component, from how much data backs it. Overall confidence is the lowest among components weighted  $\geq 0.15$ .
- Mastery update:  $\text{mastery} \leftarrow \text{mastery} + (0.15 + 0.05 \cdot \text{difficulty}) \cdot (\text{outcome} - \text{mastery})$ , where outcome is 1 if correct, 0.6 for a careless slip, 0 otherwise.

| Company    | w apt | w dsa | w cs | w proj | w comm | Bars (apt/dsa/cs/proj/comm) |
|------------|-------|-------|------|--------|--------|-----------------------------|
| TCS NQT    | .35   | .20   | .20  | .10    | .15    | 70/50/55/40/60              |
| Infosys    | .30   | .25   | .20  | .10    | .15    | 70/55/55/40/60              |
| Zoho       | .20   | .40   | .15  | .15    | .10    | 65/75/60/55/55              |
| Amazon SDE | .10   | .45   | .20  | .15    | .10    | 60/80/70/60/65              |
| Microsoft  | .05   | .45   | .20  | .15    | .15    | 60/85/70/65/70              |

Worked example (unit-tested in `lib/engine/readiness.test.ts`): scores 63/48/60/30/54 give TCS **91.7** and Amazon **69.95**, and DSA is Amazon's biggest lever (18 of the 30 missing points).

## Prompts

All in `lib/llm/prompts/`. Providers are Gemini (default: `gemini-2.5-flash`, with `gemini-flash-lite-latest` for fast calls and automatic retry/fallback on overload) and Claude, chosen by `LLM_PROVIDER`. Every call goes through `lib/llm/call.ts`, which forces structured output (tool use on Claude, JSON schema on Gemini), validates with Zod and retries once. The server-side loop has code fallbacks, so it keeps working if the AI is unavailable.

| ID                                    | Purpose                                                   | Model  | Fallback without AI       |
|---------------------------------------|-----------------------------------------------------------|--------|---------------------------|
| P1 <code>p1-resume-parser</code>      | Resume PDF → projects + checkable claims                  | Sonnet | error shown               |
| P2 <code>p2-claim-extractor</code>    | README → claims (GitHub-only projects)                    | Haiku  | error shown               |
| P3 <code>p3-repo-analyzer</code>      | Claims vs code → verdicts, evidence, interview hooks      | Sonnet | resume-only, "unverified" |
| P4 <code>p4-mistake-classifier</code> | Ambiguous wrong answers → type + explanation              | Haiku  | rules + "concept" default |
| P5 <code>p5-roadmap-generator</code>  | Sequences the code-computed budget into 2 weeks           | Sonnet | deterministic rotation    |
| P6 <code>p6-interviewer</code>        | Next project-defense question (unsupported claims first)  | Sonnet | error shown               |
| P7 <code>p7-answer-evaluator</code>   | Scores the 5 answers, overall defense score               | Sonnet | error shown               |
| P8 <code>p8-comm-scorer</code>        | Structure/relevance/clarity/length + restructured answer  | Sonnet | keyword heuristic         |
| P-GAP <code>p-gap</code>              | Must-learn, should-have, per-company expectations         | Sonnet | error shown               |
| P-CODE <code>p-code</code>            | AI code review: correctness, complexity, edge cases, hint | Sonnet | error shown               |

Clients reach an LLM only through API routes. `/api/llm/[promptId]` exposes P8 and P-CODE; the others are called inside feature routes.

## Run it

Requires Node 22.18+ and a Firebase project (Auth with Google enabled, Firestore; Storage is optional).

```
npm install
cp .env.example .env.local # fill in Firebase client + Admin values, ANTHROPIC_API_KEY,
                             GITHUB_TOKEN
npm run seed                # skills, companies, prompts, 60 questions, 2 coding problems
npm run seed:demo           # the judges' demo account with 2 weeks of history
npm run dev                 # http://localhost:3000 → "Try demo account"
```

Publish `firestore.rules` in the Firebase console (Firestore → Rules).

| Command                                       | What it does                                                   |
|-----------------------------------------------|----------------------------------------------------------------|
| <code>npm test</code>                         | Unit tests: readiness formula, mastery, mistake rules, routing |
| <code>npm run test:e2e</code>                 | The full loop against real Firestore with a throwaway user     |
| <code>npm run typecheck / lint / build</code> | Quality gates                                                  |

## Architecture

- **Next.js 16 (App Router)**, TypeScript strict, Tailwind and shadcn/ui.
- **Firebase Auth** (Google) → httpOnly session cookie → `proxy.ts` routes new users to onboarding, resumes partial ones at their saved step, and sends finished ones home.
- **Firestore via the Admin SDK only.** `lib/data/*` is a typed repository and every read is Zod-validated. The browser never writes, so students can't edit their own scores; `firestore.rules` allows only own-data reads.
- **Answer keys never leave the server.** Clients get `PublicQuestion`; grading happens in `/api/grade` and the pipeline.
- **One pipeline for every answered set** (`lib/server/pipeline.ts`): grade → attempts → rules (+P4) → mistakes → mastery → communication scoring (P8) → readiness and category snapshots.

|                         |            |                          |               |                        |              |
|-------------------------|------------|--------------------------|---------------|------------------------|--------------|
| <code>app/(auth)</code> | login      | <code>app/(app)/*</code> | sidebar pages | <code>app/api/*</code> | routes       |
| <code>lib/engine</code> | pure logic | <code>lib/server</code>  | services      | <code>lib/data</code>  | repository   |
| <code>lib/llm</code>    | LLM layer  | <code>lib/schemas</code> | Zod + types   | <code>data/seed</code> | seed content |